

# Sampling Clock Impairments

A mathematical comparison of the previous and refactored polyphase-resampling methodologies

## Purpose

Both implementations evaluate a signal through a polyphase FIR filter bank. The important change is not the interpolation itself; it is the stochastic model used to choose the sampling positions. The previous code evolves one persistent phase state by adding nominal motion, fresh jitter, and accumulated drift. The refactored code first defines a deterministic clock trajectory with fixed drift, then perturbs each output position independently with jitter.

## Main result

Previous: jitter becomes an accumulated random walk, and drift becomes a random-walk rate error that is integrated into position.

Refactored: drift is a fixed signed rate offset, while jitter is an independent displacement of the current sampling instant.

## Notation

| Symbol     | Meaning                                                                               |
|------------|---------------------------------------------------------------------------------------|
| $U$        | uprate: number of polyphase branches; $U$ phase units equal one input-sample interval |
| $D$        | drate: nominal advance per output, measured in polyphase units                        |
| $P_n$      | nominal position used for output $n$ , in polyphase units                             |
| $i_n, k_n$ | whole input index and selected polyphase branch                                       |
| $L$        | number of taps in each polyphase branch                                               |

## 1. What remains the same

The filter coefficients are partitioned into  $U$  branches. If  $h[r]$  is the prototype FIR, branch  $k$  contains coefficients whose prototype indices are congruent to  $k$  modulo  $U$ . Padding makes full FIR slices available near both signal boundaries.

$$L = \text{ceil}(\text{len}(h) / U)$$

For a selected position, the position is decomposed into a whole input-sample index  $i_n$  and a phase-bank index  $k_n$ . The output is then

$$y_n = \sum_{l=0}^{L-1} h_{k_n}[l] x_{\text{pad}}[i_n + L - 1 - l].$$

Thus, both methods use the same basic fractional-delay approximation: choose one of  $U$  discrete phases and convolve that branch with  $L$  input samples. Their disagreement lies upstream, in how the position supplied to the filter bank is generated.

### Coordinate system

A position  $Q$  in polyphase units corresponds to  $Q/U$  input-sample intervals. Euclidean decomposition gives

$$i = \text{floor}(Q/U), \quad r = Q - iU, \quad k = \text{floor}(r),$$

where  $0 \leq k \leq U - 1$ . The resolution of the phase bank is therefore  $1/U$  of an input-sample interval.

### Shared initial alignment

Both versions preserve the legacy initialization

$$P_0 = U/D.$$

When  $U = D = 32$ , this is one phase unit, or  $1/32$  of an input-sample interval. This alignment is inherited behavior; it is separate from the impairment model.

## 2. Previous methodology

The previous implementation stores the clock coordinate in  $q\_step$ . Although its name suggests an increment, it is actually the persistent position state after initialization.

```
q_step = uprate / drate
clock_drift = 0.0
```

Write  $Q_n$  for the value of  $q\_step$  used to generate output  $n$ , after carrying whole input intervals into  $input\_idx$ . The carry operation

```
while q_step >= uprate:
    q_step -= uprate
    input_idx += 1
```

changes the representation but not the absolute coordinate because

$$(i + 1)U + (q - U) = iU + q.$$

### The old stochastic update

At output  $n$ , the code draws two independent Gaussian innovations:

$$j_n \sim N(0, \sigma_j^2), \quad w_n \sim N(0, \sigma_d^2),$$

with  $\sigma_j = \text{jitter\_ppm} \times 10^{-6}$  and  $\sigma_d = \text{drift\_ppm} \times 10^{-6}$ . It then updates

$$C_n = C_{n-1} + w_n,$$
$$Q_{n+1} = Q_n + D + j_n + C_n.$$

Here  $C_n$  is the variable `clock_drift`. It is a random walk because every new drift draw is added to all previous drift draws.

```
if jitter_ppm != 0.0 or drift_ppm != 0.0:
    clock_jitter = rng.normal(0.0, jitter_std)
    clock_drift += rng.normal(0.0, drift_std)
    q_step += drate + clock_jitter + clock_drift
```

### 3. Consequences of the previous recurrence

Expanding the recurrence reveals what the random terms mean physically. Since

$$C_n = \sum_{r=0}^n w_r,$$

the position after  $n$  updates is

$$Q_n = Q_0 + nD + \sum_{r=0}^{n-1} j_r + \sum_{r=0}^{n-1} (n-r)w_r.$$

This expansion exposes two effects.

1. The old jitter is not an independent displacement. Each  $j_r$  is added to the persistent state  $Q$ , so it affects every later sampling position. The cumulative jitter term  $\sum j_r$  is a random walk, whose variance grows approximately linearly with  $n$ :

$$\text{Var}(\sum j_r) = n\sigma_j^2.$$

2. The old drift is integrated twice. The  $w_r$  draws first accumulate into the rate-like state  $C_n$ . That state is then added to position on every iteration. Early innovations receive triangular weights  $(n-r)$ , so their influence grows with time:

$$\text{Var}(\sum (n-r)w_r) = \sigma_d^2 \sum_{m=1}^n m^2 = \sigma_d^2 n(n+1)(2n+1)/6.$$

For large  $n$ , this variance is proportional to  $n^3$ . The old model therefore describes a wandering clock frequency, not a constant oscillator offset.

#### Units issue

The state  $Q$  is measured in polyphase units, but the previous standard deviations are formed directly as PPM fractions. A displacement of one input-sample period corresponds to  $U$  phase units, so a PPM-of-input-period timing displacement should be scaled by  $U$ . The old code omits this factor.

#### Interpretation

The previous implementation is defensible only if the desired phenomenon is stochastic clock wander: white innovations driving phase, plus white innovations driving a random-walk rate. It does not directly implement fixed sampling-rate drift plus independent aperture jitter.

## 4. Refactored methodology

The refactored implementation separates the nominal clock from the instantaneous timing error.

### Fixed drift

Let  $\varepsilon = \text{drift\_ppm} \times 10^{-6}$ . The nominal advance is a constant

$$\Delta = D(1 + \varepsilon).$$

The nominal clock trajectory is therefore

$$P_n = P_0 + n\Delta, \quad P_0 = U/D.$$

Positive drift increases the advance through the input and usually produces fewer output samples. Negative drift decreases it and usually produces more. No random drift draw is made: a given drift\_ppm describes one fixed oscillator-rate error.

### Independent jitter

For each output, draw an independent displacement

$$J_n \sim N(0, \sigma_J^2), \quad \sigma_J = U \cdot \text{jitter\_ppm} \times 10^{-6}.$$

The actual position used for interpolation is

$$S_n = \text{clip}(P_n + J_n, 0, S_{\max}).$$

Crucially,  $J_n$  is not used in the recurrence for  $P_{n+1}$ . Hence

$$P_{n+1} = P_n + \Delta$$

regardless of the current jitter realization. The jitter variance at each sample remains  $\sigma_J^2$ ; it does not grow with  $n$ .

### Position decomposition

The refactor uses `divmod(sample_position, uprate)`. In mathematics:

$$i_n = \text{floor}(S_n/U), \quad r_n = S_n - i_n U, \quad k_n = \text{floor}(r_n).$$

This directly converts one absolute coordinate into the integer input index and fractional-delay branch, removing the manual phase-carry state.

## 5. Side-by-side comparison

| Feature                   | Previous                                                                  | Refactored                                                |
|---------------------------|---------------------------------------------------------------------------|-----------------------------------------------------------|
| Nominal clock             | Mixed into $q\_step$ with random terms                                    | $P_n = P_0 + nD(1+\epsilon)$                              |
| Drift meaning             | Gaussian innovations accumulate into a random-walk rate                   | Fixed signed fractional rate error                        |
| Jitter meaning            | Fresh innovation added to persistent position; effects accumulate         | Independent displacement of current output only           |
| Jitter variance over time | Position contribution grows as $n\sigma_j^2$                              | Constant $\sigma_j^2$ per output                          |
| Drift-position variance   | Approximately proportional to $n^3$                                       | Zero for fixed input parameters                           |
| PPM scaling               | No factor $U$                                                             | Timing jitter multiplied by $U$ phase units/sample        |
| State representation      | Integer index plus mutable residual phase                                 | One absolute nominal position                             |
| Boundary behavior         | Upper endpoint clamped after decomposition; no explicit lower-state model | Complete jittered position clipped to representable range |
| Physical model            | Clock wander / stochastic frequency noise                                 | Constant frequency offset + aperture jitter               |

### A short numerical picture

Take  $U = D = 32$ . Then  $P_0 = 1$  phase unit. With  $\text{drift\_ppm} = 1000$ , the refactored increment is  $\Delta = 32.032$ . Ignoring jitter, the positions are 1, 33.032, 65.064, ... . A jitter draw changes only the position used on that one output.

In the previous method, a jitter draw of +0.01 changes  $q\_step$  by +0.01, so every later position inherits that shift unless later draws happen to cancel it. A positive drift innovation also remains in  $\text{clock\_drift}$  and is added again on each later iteration.

## 6. Statistical interpretation

The two implementations correspond to different stochastic processes.

### Previous model

$$Q_{n+1} - Q_n = D + j_n + C_n, \quad C_n - C_{n-1} = w_n.$$

This is a two-state model: position is driven by white noise  $j_n$ , and its increment is additionally driven by the integrated noise  $C_n$ . It can emulate phase wander and random-walk frequency noise, but the parameters called `jitter_ppm` and `drift_ppm` no longer have the simple meanings stated by the API.

### Refactored model

$$S_n = P_0 + nD(1+\varepsilon) + J_n.$$

This is linear deterministic motion plus white timing noise. Its expected value and variance, away from clipping boundaries, are

$$E[S_n] = P_0 + nD(1+\varepsilon), \quad \text{Var}(S_n) = \sigma_j^2.$$

For  $m \neq n$ , the jitter contributions are independent, so  $\text{Cov}(J_m, J_n) = 0$ . This matches the documented claim that jitter does not accumulate into later nominal positions.

### Clipping caveat

Near the first and last representable positions, clipping truncates the Gaussian displacement. Consequently, the actual boundary distribution is no longer exactly zero-mean Gaussian. This is a deliberate finite-buffer rule, not part of the ideal interior clock model.

### Phase quantization caveat

The continuous fractional remainder  $r_n$  is converted to  $k_n = \text{floor}(r_n)$ . Therefore, the requested timing position is quantized downward to one of  $U$  branches. The maximum phase-selection error is less than one phase unit, or less than  $1/U$  input-sample interval.

## 7. Bottom line

The refactor changes the impairment definition, not merely the organization of the loop.

The previous method generates

$$Q_n = Q_0 + nD + \text{accumulated jitter} + \text{integrated random-walk drift.}$$

The refactored method generates

$$S_n = P_0 + nD(1+\varepsilon) + \text{independent jitter.}$$

Use the refactored model when the API is intended to mean:

- `drift_ppm`: a fixed signed sampling-rate mismatch;
- `jitter_ppm`: RMS independent timing error measured relative to one input-sample period.

Use a state-space or phase-noise model resembling the previous recurrence only when clock wander is intentional. In that case, the random processes should be named and parameterized explicitly - for example, white phase noise, random-walk phase noise, and random-walk frequency noise - rather than calling them simply jitter and drift.

### Compact implementation map

```
nominal_position_increment = drate * (1 + drift_ppm * 1e-6)
corresponds to  $\Delta = D(1+\varepsilon)$ .
```

```
timing_offset ~ Normal(0, uprate * jitter_ppm * 1e-6)
corresponds to  $J_n \sim N(0, \sigma_J^2)$ .
```

```
sample_position = clip(nominal_position + timing_offset, ...)
corresponds to  $S_n = \text{clip}(P_n + J_n, \dots)$ .
```

```
nominal_position += nominal_position_increment
corresponds to  $P_{n+1} = P_n + \Delta$ , deliberately excluding  $J_n$ .
```